

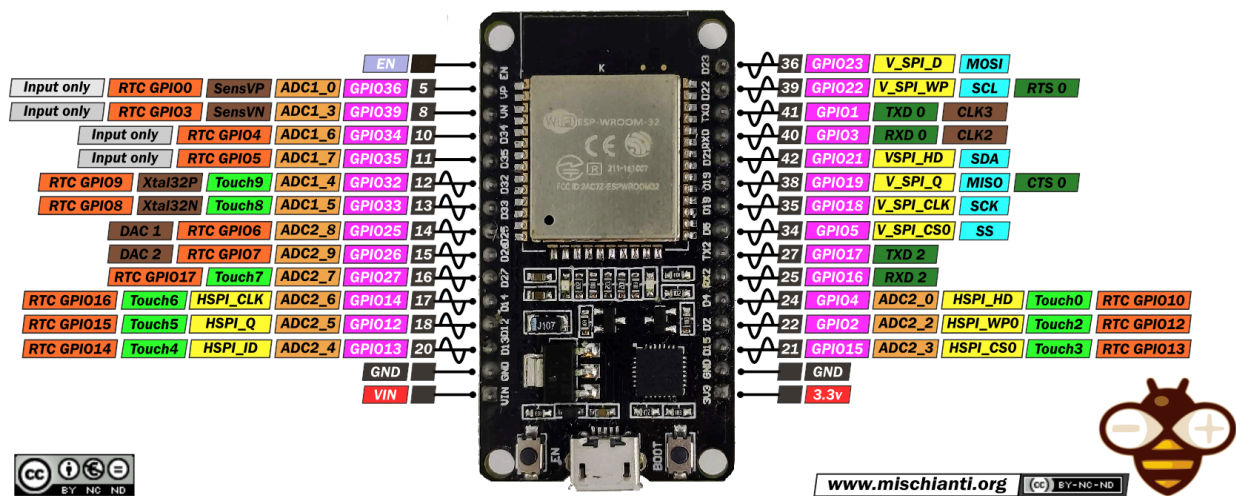
STR

Memorias Prácticas FreeRTOS y ESP32: Práctica 1

Judith Rot Alcántara

Miguel Ángel Corredor Ferrer

ESP32 DEV KIT V1 PINOUT



www.mischianti.org (CC) BY-NC-ND



Practica 1

Parte 1

Apartado I: Primera compilación

1.- ¿Cuánta capacidad tiene la flash del hardware que estás utilizando?

La capacidad de la flash del hardware nos aparece al compilar nuestro programa. En este caso sería 4MB. A continuación una captura que lo respalda.

```
Verbose mode can be enabled via `-v, --verbose` option
CONFIGURATION: https://docs.platformio.org/page/boards/espressif32/esp32dev.html
PLATFORM: Espressif 32 (6.5.0) > Espressif ESP32 Dev Module
HARDWARE: ESP32 240MHz, 320KB RAM, 4MB Flash
DEBUG: Current (cmsis-dap) External (cmsis-dap, esp-bridge, esp-prog, iot-bus-jtag, jlink, minimodule, olimex-arm-usb-ocd, olimex-arm-usb-ocd-h, olimex-arm-usb-tiny-h, olimex-jtag-tiny, tumpa)
PACKAGES:
```

2.- ¿Qué versión de espidf tiene el entorno que ha creado PlatformIO?

La versión es la 5.1.2

```
PACKAGES:
- framework-espidf @ 3.50102.240122 (5.1.2)
- tool-cmake @ 3.16.4
- tool-esptoolpy @ 1.40501.0 (4.5.1)
- tool-idf @ 1.0.1
- tool-mconf @ 1.4060000.20190628 (406.0.0)
- tool-ninja @ 1.9.0
- tool-riscv32-esp-elf-gdb @ 11.2.0+20220823
- tool-xtensa-esp-elf-gdb @ 11.2.0+20230208
- toolchain-esp32ulp @ 1.23500.220830 (2.35.0)
- toolchain-xtensa-esp32 @ 12.2.0+20230208
```

3.- ¿Cuánta memoria Flash ocupa el firmware generado? ¿Qué porcentaje supone del disponible?

El firmware generado ocupa 174997 bytes, lo cual supone un 16,7% usado.

```
Flash: [== ] 16.7% (used 174997 bytes from 1048576 bytes)
```

4.- ¿Cuánta memoria RAM supone la ejecución de este código? ¿Qué porcentaje supone del disponible?

La ejecución del código supone 10588 bytes, un 3,2% usado.

```
RAM: [ ] 3.2% (used 10588 bytes from 327680 bytes)
```

5.- ¿En qué ruta está el fichero .elf generado?

En la ruta .pio\build\esp32dev\firmware.elf

```
Retrieving maximum program size .pio\build\esp32dev\firmware.elf
Checking size .pio\build\esp32dev\firmware.elf
```

Apartado II: Primera compilación

1.- Haz una captura de pantalla de la salida por monitor que te da la aplicación.

```
ES[0;32mI (318) main_task: Calling app_main()ES[0m
ES[0;32mI (318) P1_1: Hello ESP32!ES[0m
ES[0;32mI (318) P1_1: Miguel Angel, Corredor FerrerES[0m
ES[0;32mI (328) P1_1: Miguel Angel, Corredor FerrerES[0m
ES[0;32mI (328) P1_1: Modelo: ESP32, Cores: 2, Frecuencia: 160000000ES[0m
ES[0;32mI (338) main_task: Returned from app_main()ES[0m
```

2.- ¿Qué tipo de información te ofrece el log de inicio del sistema?

La información que ofrece sería: (revisión del chip, partición de memoria)

```

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:7076
load:0x40078000,len:15584
ho 0 tail 12 room 4
load:0x40080400,len:4
load:0x40080404,len:3876
entry 0x4008064c
ES[0;32mI (31) boot: ESP-IDF 5.1.2 2nd stage bootloaderES[0m
ES[0;32mI (31) boot: compile time Mar  8 2024 10:47:32ES[0m
ES[0;32mI (31) boot: Multicore bootloaderES[0m
ES[0;32mI (35) boot: chip revision: v3.0ES[0m
ES[0;32mI (39) boot.esp32: SPI Speed      : 40MHzES[0m
ES[0;32mI (44) boot.esp32: SPI Mode      : DIOES[0m
ES[0;32mI (48) boot.esp32: SPI Flash Size : 4MBES[0m
ES[0;32mI (58) boot: Partition Table:ES[0m
ES[0;32mI (62) boot: ## Label                Usage            Type ST Offset   LengthES[0m
ES[0;32mI (69) boot:  0 nvs                    WiFi data        01 02 00009000 00006000ES[0m
ES[0;32mI (77) boot:  1 phy_init                RF data          01 01 0000f000 00001000ES[0m
ES[0;32mI (84) boot:  2 factory                  factory app       00 00 00010000 00100000ES[0m
ES[0;32mI (92) boot: End of partition tableES[0m
ES[0;32mI (96) esp_image: segment 0: paddr=00010020 vaddr=3f400020 size=09318h ( 37656) mapES[0m
ES[0;32mI (118) esp_image: segment 1: paddr=00019340 vaddr=3ffb0000 size=020e4h ( 8420) loadES[0m
ES[0;32mI (121) esp_image: segment 2: paddr=0001b42c vaddr=40080000 size=04bech ( 19436) loadES[0m
ES[0;32mI (132) esp_image: segment 3: paddr=00020020 vaddr=400d0020 size=136ech ( 79596) mapES[0m
ES[0;32mI (161) esp_image: segment 4: paddr=00033714 vaddr=40084bec size=075c4h ( 30148) loadES[0m
ES[0;32mI (180) boot: Loaded app from partition at offset 0x10000ES[0m
ES[0;32mI (180) boot: Disabling RNG early entropy source...ES[0m
ES[0;32mI (191) cpu_start: Multicore appES[0m
ES[0;32mI (192) cpu_start: Pro cpu up.ES[0m
ES[0;32mI (192) cpu_start: Starting app cpu, entry point is 0x40081220ES[0m
ES[0;32mI (0) cpu_start: App cpu up.ES[0m

```

Parte 2

Apartado III: Introducción a Kconfig.

1.- Navega a la opción de Menuconfig “Bootloader config” -> “Bootloader log verbosity”.
¿Cómo está configurada esta característica? ¿Para qué sirve? (apóyate en la información que te ofrece el comando “?”, aporta captura de pantalla)

La opción "Bootloader log verbosity" en el menú de configuración (Menuconfig) del ESP-IDF se refiere al nivel de detalle de los registros (logs) generados por el cargador de arranque (bootloader) durante el inicio del dispositivo. Los niveles de verbosidad disponibles suelen ser:

- None: Sin registros de arranque.
- Error: Solo mensajes de error.
- Warning: Mensajes de advertencia y errores.
- Info: Información básica sobre el proceso de arranque.
- Debug: Información detallada para depurar problemas.

```
(Top) → Bootloader config → Bootloader log verbosity
Espressif IoT Development Framework Configuration

( ) No output
( ) Error
( ) Warning
(X) Info
( ) Debug
( ) Verbose

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

Choice information
Name: BOOTLOADER_LOG_LEVEL
Prompt: Bootloader log verbosity
Type: bool
Mode: y
Help:
    Specify how much output to see in bootloader logs.
Choice symbols:
- BOOTLOADER_LOG_LEVEL_NONE
- BOOTLOADER_LOG_LEVEL_ERROR
- BOOTLOADER_LOG_LEVEL_WARN
↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[ESC/q] Return to menu  [/] Jump to symbol
```

```

↑↑↑↑↑↑↑↑↑↑↑↑ Choice information
Choice symbols:
- BOOTLOADER_LOG_LEVEL_NONE
- BOOTLOADER_LOG_LEVEL_ERROR
- BOOTLOADER_LOG_LEVEL_WARN
- BOOTLOADER_LOG_LEVEL_INFO (selected)
- BOOTLOADER_LOG_LEVEL_DEBUG
- BOOTLOADER_LOG_LEVEL_VERBOSE

Default:
- BOOTLOADER_LOG_LEVEL_INFO

Kconfig definition, with parent deps. propagated to 'depends on'
↓↓↓↓↓↓↓↓↓↓↓↓
[ESC/q] Return to menu [/] Jump to symbol

Symbol information
Name: BOOTLOADER_LOG_LEVEL_INFO
Prompt: Info
Type: bool
Value: y

Direct dependencies (=y):
<choice BOOTLOADER_LOG_LEVEL>

Kconfig definition, with parent deps. propagated to 'depends on'
=====

At C:/Users/migue/.platformio/packages/framework-esp8266/components/bootloader/Kconfig.projbuild:48
Included via C:/Users/migue/.platformio/packages/framework-esp8266/Kconfig:261 -> C:/Users/migue/Documents/PlatformIO/Projects/STR2024
↓↓↓↓↓↓↓↓↓↓↓↓

Kconfig definition, with parent deps. propagated to 'depends on'
=====

At C:/Users/migue/.platformio/packages/framework-esp8266/components/bootloader/Kconfig.projbuild:36
Included via C:/Users/migue/.platformio/packages/framework-esp8266/Kconfig:261 -> C:/Users/migue/Documents/PlatformIO/Projects/STR2024
Menu path: (Top) -> Bootloader config

choice BOOTLOADER_LOG_LEVEL
bool "Bootloader log verbosity"
default BOOTLOADER_LOG_LEVEL_INFO(=y)
help
Specify how much output to see in bootloader logs.

[ESC/q] Return to menu [/] Jump to symbol

```

2.- Explora las opciones de la tabla de particiones. Haz una pequeña investigación en internet sobre el término OTA. ¿Qué es y para qué sirve?

La sigla en inglés "OTA" (Over-The-Air) se refiere a actualización por aire y es un método inalámbrico de provisión de nuevo software o firmware a los teléfonos móviles y a las tabletas. Una actualización OTA puede ser automática o manual.

```
(Top) → Partition Table
Espressif IoT Development Framework Configuration
Partition Table (Single factory app, no OTA) -->
(0x8000) Offset of partition table
[*] Generate an MD5 checksum for the partition table

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

(Top) → Partition Table → Partition Table
Espressif IoT Development Framework Configuration
(X) Single factory app, no OTA
( ) Single factory app (large), no OTA
( ) Factory app, two OTA definitions
( ) Custom partition table CSV

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

Symbol information
Name: PARTITION_TABLE_SINGLE_APP
Prompt: Single factory app, no OTA
Type: bool
Value: y
Help:
This is the default partition table, designed to fit into a 2MB or
larger flash with a single 1MB app partition.
The corresponding CSV file in the IDF directory is
components/partition_table/partitions_singleapp.csv
↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[ESC/q] Return to menu [/] Jump to symbol

↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑ Symbol information
Help:
This is the default partition table, designed to fit into a 2MB or
larger flash with a single 1MB app partition.
The corresponding CSV file in the IDF directory is
components/partition_table/partitions_singleapp.csv
This partition table is not suitable for an app that needs OTA
(over the air update) capability.
Direct dependencies (=y):
<choice PARTITION_TABLE_TYPE>
↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[ESC/q] Return to menu [/] Jump to symbol
```

```

↑↑↑↑↑↑↑↑↑↑↑↑↑↑      Symbol information
Direct dependencies (=y):
  <choice PARTITION_TABLE_TYPE>

Kconfig definition, with parent deps. propagated to 'depends on'
=====

At C:/Users/migue/.platformio/packages/framework-esp8266/components/partition_table/Kconfig.projbuild:17
Included via C:/Users/migue/.platformio/packages/framework-esp8266/Kconfig:261 -> C:/Users/migue/Documents/PlatformIO/Projects/STR2024
Menu path: (Top) -> Partition Table -> Partition Table

config PARTITION_TABLE_SINGLE_APP
  bool "Single factory app, no OTA"
  depends on <choice PARTITION_TABLE_TYPE>
↓↑↑↑↑↑↑↑↑↑↑↑↑↑
[ESC/q] Return to menu      [/] Jump to symbol

```

```

↑↑↑↑↑↑↑↑↑↑↑↑↑↑      Symbol information

config PARTITION_TABLE_SINGLE_APP
  bool "Single factory app, no OTA"
  depends on <choice PARTITION_TABLE_TYPE>
  help
    This is the default partition table, designed to fit into a 2MB or
    larger flash with a single 1MB app partition.

    The corresponding CSV file in the IDF directory is
    components/partition_table/partitions_singleapp.csv

    This partition table is not suitable for an app that needs OTA
    (over the air update) capability.
[ESC/q] Return to menu      [/] Jump to symbol

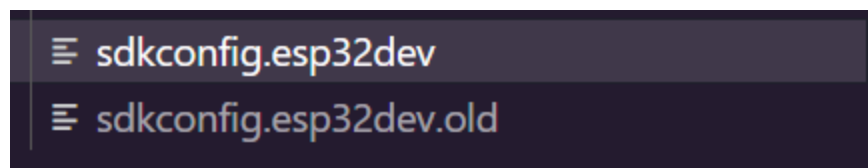
```

Apartado IV: Primer Kconfig.projbuild.

1.- Comprueba el estado de sdkconfig.esp32dev antes y después de entrar en Menuconfig y activar el soporte para Bluetooth ("Component config"-> "Bluetooth"). ¿Qué ha cambiado?.

Para activar la configuración, usamos Component config y seleccionamos la opción Bluetooth.

Al activar dicha opción se genera un fichero `sdkconfig.esp32dev`, mientras que el otro pasa a llamarse `sdkconfig.esp32dev.old`. La principal diferencia es que ahora en este fichero hay toda una nueva configuración para Bluetooth. Lo vemos en las siguientes imágenes.



Esta sería la configuración antigua:


```
# Bluetooth
#
# CONFIG_BT_ENABLED is not set
# end of Bluetooth

#
# Driver Configurations
#
#
```

Y esta la nueva con Bluetooth activado.

```
# Bluetooth
#
CONFIG_BT_ENABLED=y
CONFIG_BT_BLUEDROID_ENABLED=y
# CONFIG_BT_NIMBLE_ENABLED is not set
# CONFIG_BT_CONTROLLER_ONLY is not set
CONFIG_BT_CONTROLLER_ENABLED=y
# CONFIG_BT_CONTROLLER_DISABLED is not set

#
# Bluedroid Options
#
CONFIG_BT_BTC_TASK_STACK_SIZE=3072
CONFIG_BT_BLUEDROID_PINNED_TO_CORE_0=y
# CONFIG_BT_BLUEDROID_PINNED_TO_CORE_1 is not set
CONFIG_BT_BLUEDROID_PINNED_TO_CORE=0
CONFIG_BT_BTU_TASK_STACK_SIZE=4096
# CONFIG_BT_BLUEDROID_MEM_DEBUG is not set
# CONFIG_BT_CLASSIC_ENABLED is not set
```

2.- Compila de nuevo el firmware con la modificación. ¿Ha cambiado el tamaño del binario?

Ha cambiado el tamaño del fichero, pues el original ocupaba alrededor de 175K y el que tiene activada la configuración Bluetooth 200K. A continuación capturas que lo demuestran.

Fichero original :

```
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [          ] 3.2% (used 10588 bytes from 327680 bytes)
Flash: [==       ] 16.7% (used 175033 bytes from 1048576 bytes)
Building .pio\build\esp32dev\firmware.bin
esptool.py v4.5.1
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
```

Fichero modificado:

```
Checking size .pio\build\esp32dev\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [          ] 4.7% (used 15428 bytes from 327680 bytes)
Flash: [==       ] 19.1% (used 200133 bytes from 1048576 bytes)
Building .pio\build\esp32dev\firmware.bin
esptool.py v4.5.1
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.
```

Apartado IV: Trabajo con menuconfig

1.- Modifica el fichero Kconfig.projbuild para crear un ejemplo de cada tipo de menú que permite la herramienta. Explora Utiliza las diferentes configuraciones en tu código usando printf. Haz capturas de pantalla.

Antes de eso configuramos la actividad 4, donde ponemos en Kconfig un menú con nuestros nombres, lanzamos menuconfig.

```
(Top)
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑      Espressif IoT Development Framework Configuration
Bootloader config --->
Security features --->
Application manager --->
Serial flasher config --->
Partition Table --->
Configuración Práctica 1 Parte 2 --->
Compiler options --->
Component config --->
[ ] Make experimental features visible

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                   [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

(Top) → Configuración Práctica 1 Parte 2
Espressif IoT Development Framework Configuration
(Miguel) Nombre1
(Corredor) Apellidos1
(Judith) Nombre2
(Rot) Apellidos2

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                   [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Si ahora compilamos no nos da error, pues ha dado tiempo de generar las macros. De hecho si observamos nuestro fichero sdkconfig.esp32dev ahora las tiene definidas con nuestros nombres.

```
#
# Configuración Práctica 1 Parte 2
#
CONFIG_NOMBRE1="Miguel"
CONFIG_APELLIDOS1="Corredor"
CONFIG_NOMBRE2="Judith"
CONFIG_APELLIDOS2="Rot"
# end of Configuración Práctica 1 Parte 2

#
# Compiler options
```

Aparte de este tipo de menú hemos investigado otros tipos y hemos añadido el código a nuestro archivo Kconfig.

Código kconfig

menu "Mi Menu de Configuración"

Este bloque permite elegir entre a, b, o c si MI_OPCION_ABC está habilitado

config MI_OPCION_ABC

tristate "Elige una opción (a, b, c)"

default y

help

Elige entre las opciones a, b o c.

if MI_OPCION_ABC

choice

prompt "Elige tu opción"

default OPCION_A

config OPCION_A

bool "Opción A"

config OPCION_B

bool "Opción B"

config OPCION_C

bool "Opción C"

endchoice

endif

Submenu para opciones adicionales

menu "Submenu Ejemplo"

config MI_SUBMENU_OPCION

bool "Una opción en submenu"

default y

help

Esta es una opción dentro de un submenu.

```
config MI_SUBMENU_OPCION_NUMERICA
  int "Configura un valor numérico"
  default 5
  range 1 10
  help
    Configura un valor entre 1 y 10.
endmenu

endmenu
```

Código main

```
#include "stdio.h"
#include "sdkconfig.h"

void app_main() {
    // Verifica cuál de las opciones ha sido seleccionada (si MI_OPCION_ABC está habilitado)
    #ifdef CONFIG_MI_OPCION_ABC
    #if defined(CONFIG_OPCION_A)
    printf("Has seleccionado la Opción A\n");
    #elif defined(CONFIG_OPCION_B)
    printf("Has seleccionado la Opción B\n");
    #elif defined(CONFIG_OPCION_C)
    printf("Has seleccionado la Opción C\n");
    #endif
    #else
    printf("La selección de opciones ABC está deshabilitada.\n");
    #endif

    // Imprime si la opción en el submenu está activada
    #ifdef CONFIG_MI_SUBMENU_OPCION
    printf("La opción en submenu está activada.\n");
    #endif

    // Muestra el valor numérico configurado
    printf("El valor numérico configurado es: %d\n", CONFIG_MI_SUBMENU_OPCION_NUMERICA);
}
```

De esta forma al lanzar menuconfig, nos saldrían los tres tipos nuevos que hemos añadido:

```
(Top)
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑      Espressif IoT Development Framework Configuration
Serial flasher config --->
Partition Table --->
Mi Menu de Configuración --->
Compiler options --->
Component config --->
[ ] Make experimental features visible

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

1-. Menú para seleccionar una entre varias opciones:

```
(Top) → Mi Menu de Configuración → Elige una opción (a, b, c) → Elige tu opción
Espressif IoT Development Framework Configuration
( ) Opción A
(X) Opción B
( ) Opción C

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                   [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Dentro del Submenú ejemplo encontramos los otros dos ejemplos.

2-. Menú para marcar o desmarcar una única opción.

3-. Menú para escribir un número entre 1-10.

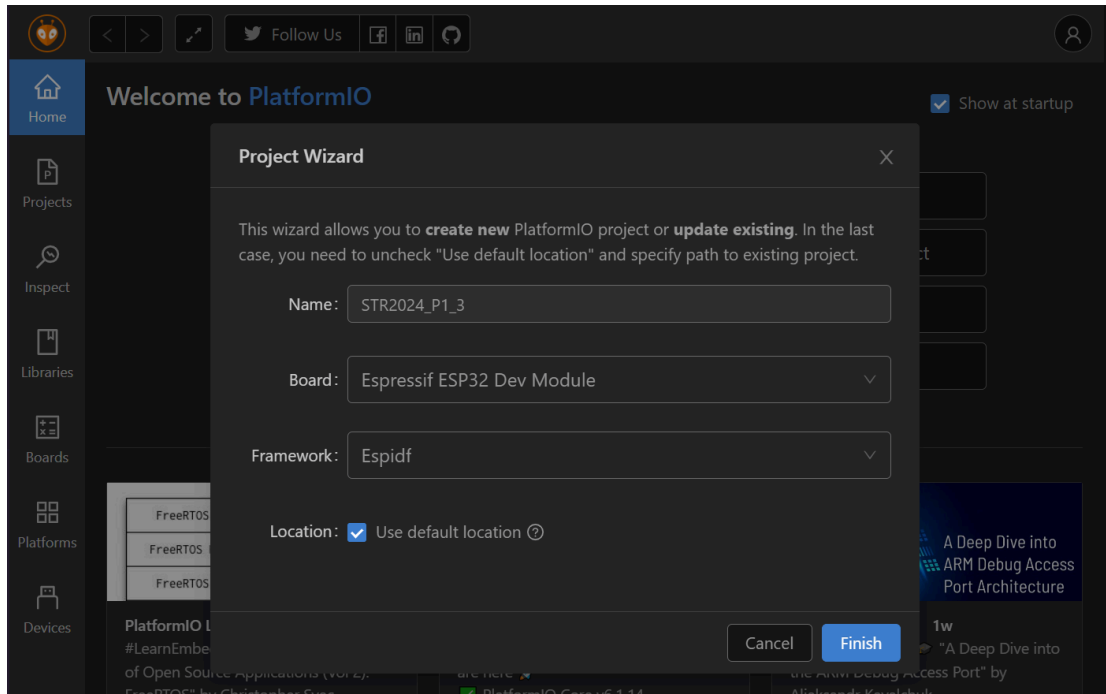
```
(Top) → Mi Menu de Configuración → Submenu Ejemplo
Espressif IoT Development Framework Configuration
[*] Una opción en submenu
(6) Configura un valor numérico
```

Compilamos y cargamos el firmware en la placa. Una vez hecho esto, reseteamos y vemos que se muestran las opciones elegidas en la salida en el monitor:

```
es[0;32mI (317) main_task: Calling app_main()es[0m
Has seleccionado la Opción B
La opción en submenu está activada.
El valor numérico configurado es: 5
es[0;32mI (327) main_task: Returned from app_main()es[0m
```

Parte 3

Empezamos creando nuestro nuevo proyecto STR2024_P1_3 y modificaremos el fichero platformio.ini para que tenga dos configuraciones: polling e interrupt.



Añadimos el código correspondiente y ejecutamos Menuconfig para configurar el PIN que tenemos conectado (en nuestro caso el 13)

```
#  
# Configuración Práctica 1 parte 3  
#  
CONFIG_GPIO_PIN=13  
# end of Configuración Práctica 1 parte 3
```

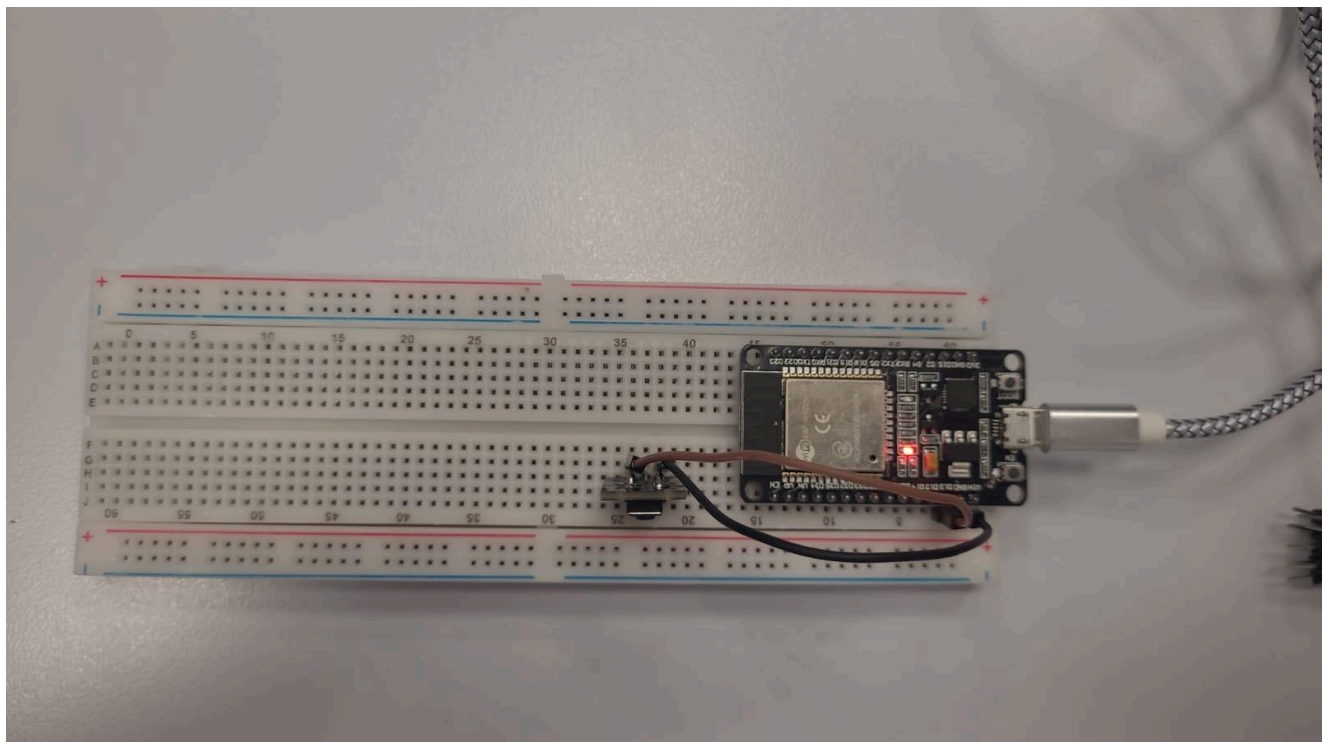
```
(Top) → Configuración Práctica 1 parte 3
Espressif IoT Development Framework Configuration
(13) GPIO Pin

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                  [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

Guardamos para actualizar el valor.

Apartado V: Polling

Montamos el circuito quedando así:



1.- Compila y carga el firmware de la configuración polling desde Platformio. Utiliza la opción “Monitor” para visualizar los mensajes de la aplicación en ejecución. Pulsa el botón varias veces. Aporta capturas de pantalla.

Si vemos los mensajes de “Monitor” vemos que cada vez que pulsamos el botón aparece en terminal:

```
ES[0;32mI (0) cpu_start: App cpu up.ES[0m
ES[0;32mI (213) cpu_start: Pro cpu start user codeES[0m
ES[0;32mI (213) cpu_start: cpu freq: 160000000 HzES[0m
ES[0;32mI (213) cpu_start: Application information:ES[0m
ES[0;32mI (218) cpu_start: Project name: STR2024_P1_3ES[0m
ES[0;32mI (223) cpu_start: App version: 1ES[0m
ES[0;32mI (227) cpu_start: Compile time: Apr 12 2024 10:53:52ES[0m
ES[0;32mI (233) cpu_start: ELF file SHA256: a6212a761a444388...ES[0m
ES[0;32mI (239) cpu_start: ESP-IDF: 5.1.2ES[0m
ES[0;32mI (244) cpu_start: Min chip rev: v0.0ES[0m
ES[0;32mI (249) cpu_start: Max chip rev: v3.99 ES[0m
ES[0;32mI (254) cpu_start: Chip rev: v3.0ES[0m
ES[0;32mI (259) heap_init: Initializing. RAM available for dynamic allocation:ES[0m
ES[0;32mI (266) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAMES[0m
ES[0;32mI (272) heap_init: At 3FFB29D0 len 0002D630 (181 KiB): DRAMES[0m
ES[0;32mI (278) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAMES[0m
ES[0;32mI (284) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAMES[0m
ES[0;32mI (291) heap_init: At 4008C1B0 len 00013E50 (79 KiB): IRAMES[0m
ES[0;32mI (298) spi_flash: detected chip: genericES[0m
ES[0;32mI (302) spi_flash: flash io: dioES[0m
ES[0;32mI (306) app_start: Starting scheduler on CPU0ES[0m
ES[0;32mI (311) app_start: Starting scheduler on CPU1ES[0m
```

```

ES[0;32mI (0) cpu_start: App cpu up. ES[0m
ES[0;32mI (213) cpu_start: Pro cpu start user code ES[0m
ES[0;32mI (213) cpu_start: cpu freq: 160000000 Hz ES[0m
ES[0;32mI (213) cpu_start: Application information: ES[0m
ES[0;32mI (218) cpu_start: Project name: STR2024_P1_3 ES[0m
ES[0;32mI (223) cpu_start: App version: 1 ES[0m
ES[0;32mI (227) cpu_start: Compile time: Apr 12 2024 10:53:52 ES[0m
ES[0;32mI (233) cpu_start: ELF file SHA256: a6212a761a444388... ES[0m
ES[0;32mI (239) cpu_start: ESP-IDF: 5.1.2 ES[0m
ES[0;32mI (244) cpu_start: Min chip rev: v0.0 ES[0m
ES[0;32mI (249) cpu_start: Max chip rev: v3.99 ES[0m
ES[0;32mI (254) cpu_start: Chip rev: v3.0 ES[0m
ES[0;32mI (259) heap_init: Initializing. RAM available for dynamic allocation: ES[0m
ES[0;32mI (266) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM ES[0m
ES[0;32mI (272) heap_init: At 3FFB29D0 len 0002D630 (181 KiB): DRAM ES[0m
ES[0;32mI (278) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM ES[0m
ES[0;32mI (284) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM ES[0m
ES[0;32mI (291) heap_init: At 4008C1B0 len 00013E50 (79 KiB): IRAM ES[0m
ES[0;32mI (298) spi_flash: detected chip: generic ES[0m
ES[0;32mI (302) spi_flash: flash io: dio ES[0m
ES[0;32mI (306) app_start: Starting scheduler on CPU0 ES[0m
ES[0;32mI (311) app_start: Starting scheduler on CPU1 ES[0m

```

2.- Como habrás observado, el código asociado al botón se ejecuta continuamente en bucle. Modifica el código para que únicamente se ejecute una vez por pulsación. Incluye tu código, su explicación, y capturas de pantalla en la memoria.

El código nos quedaría:

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

const char* TAG = "P1_3";

void app_main(void)
{
    // Configuración inicial del GPIO
    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_DISABLE,
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << CONFIG_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en = GPIO_PULLUP_ENABLE
    };
    gpio_config(&io_conf);

    uint8_t gpio_value;
    bool was_pressed = false; // Estado para controlar si el botón ha sido previamente presionado

    while (1)
    {
        gpio_value = gpio_get_level(CONFIG_GPIO_PIN);

        if (gpio_value == 0 && !was_pressed)
        {
            // Solo entra si el botón está presionado y no fue registrado antes
            ESP_LOGI(TAG, "Botón pulsado!");
            was_pressed = true; // Marca el botón como presionado
        }
        else if (gpio_value == 1)
        {
            // Reinicia el estado cuando el botón se libera
            was_pressed = false;
        }

        vTaskDelay(10 / portTICK_PERIOD_MS);
    }
}
return go(f, seed, [])
}

```

Las modificaciones que hemos hecho en dicha implementación es:

Explicación del código:

1. **Estado `was_pressed`:** Utilizamos una variable booleana `was_pressed` para rastrear si una pulsación del botón ya ha sido detectada. Esto previene que múltiples eventos de pulsación sean registrados mientras el botón sigue presionado.
2. **Detección de la pulsación:** Cuando detectamos que el nivel del GPIO es `0` (lo cual indica que el botón está siendo presionado) y `was_pressed` es `false` (indicando que es una nueva pulsación), registramos el evento y cambiamos `was_pressed` a `true`.
3. **Reinicio del estado:** Si el botón se libera (el GPIO cambia a `1`), reseteamos `was_pressed` a `false`, permitiendo que una nueva pulsación pueda ser detectada.

Para el siguiente apartado creamos el main de interrupt y añadimos el código facilitado.

Apartado VI: Interrupt

1.- Compila y carga el firmware de la configuración interrupts desde Platformio. Utiliza la opción "Monitor" para visualizar los mensajes de la aplicación en ejecución. Pulsa el botón. ¿Qué ha pasado? Aporta capturas de pantalla.

Lo que ha pasado es que al poner ESP_LOGI nos encontramos ante un método bloqueante para una rutina ISR, y se produce un reseteo.

```

abort() was called at PC 0x40082feb on core 0

Backtrace: 0x40081d2e:0x3ffb0910 0x4008599d:0x3ffb0930 0x4008aeba:0x3ffb0950 0x40082feb:0x3ffb09c0 0x40083129:0x3ffb09f0 0x400831a2:0x3ffb0a10 0x400d775e:0x3ffb0a40 0x400da8cd:0x3ffb0d60 0x400e4a51:0x3ffb0d90 0x4008ad6d:0x3ffb0dc0 0x400810ad:0x3ffb0e10 0x40081176:0x3ffb0e30 0x40081206:0x3ffb0e60 0x40081329:0x3ffb0e90 0x400828ed:0x3ffb0eb0 0x40084843:0x3ffb52a0 0x400d3947:0x3ffb52c0 0x400868ea:0x3ffb52e0 0x40087e3d:0x3ffb5300

ELF file SHA256: 0146d65a0b31fee1

Rebooting...
ets Jul 29 2019 12:21:46

rst:0xc (SW_CPU_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:7076
load:0x40078000,len:15584
ho 0 tail 12 room 4
load:0x40080400,len:4
load:0x40080404,len:3876
entry 0x4008064c
es[0;32mI (31) boot: ESP-IDF 5.1.2 2nd stage bootloaderes[0m
es[0;32mI (31) boot: compile time Apr 12 2024 11:07:03es[0m
es[0;32mI (31) boot: Multicore bootloaderes[0m

```

2.- Dentro del código de la interrupción, sustituye la macro ESP_LOGI por ESP_DRAM_LOGI. Vuelve a compilar y cargar el firmware. ¿Por qué hemos tenido que hacer este cambio? Busca información en Internet.

```

static void IRAM_ATTR gpio_isr_handler(void* arg)
{
    ESP_DRAM_LOGI(TAG, "Botón pulsado!");
}

```

Ahora al pulsar el botón no nos sale ningún error e imprime "Botón pulsado!" al pulsarlo.

```
ES[0;32mI (292) heap_init: At 4008C370 len 00013C90 (79 KiB): IRAMES[0m
ES[0;32mI (299) spi_flash: detected chip: genericES[0m
ES[0;32mI (303) spi_flash: flash io: dioES[0m
ES[0;32mI (307) app_start: Starting scheduler on CPU0ES[0m
ES[0;32mI (312) app_start: Starting scheduler on CPU1ES[0m
ES[0;32mI (312) main_task: Started on CPU0ES[0m
ES[0;32mI (322) main_task: Calling app_main()ES[0m
ES[0;32mI (322) gpio: GPIO[13]| InputEn: 1| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:2 ES[0m
I P1_3: Botón pulsado!
I P1_3: Botón pulsado!
I P1_3: Botón pulsado!
I P1_3: Botón pulsado!
I P1_3: Botón pulsado!
I P1_3: Botón pulsado!
```

Esto se debe a que hay estructuras que no se pueden usar al ejecutar una rutina ISR. Este es el caso de la macro **ESP_LOGI**.

Apartado VII: GPIO como salida

1.- En el pin GPIO 2 tienes un LED soldado en la placa. Modifica el código `src/polling/main.c` para que este led se apague cuando el botón se pulsa, y se encienda en caso contrario. Aporta código, explicación y capturas en la memoria

Modificamos el código de forma que nos queda:


```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

const char* TAG = "P1_3";
#define LED_GPIO_PIN 2 // Definición del pin del LED

void app_main(void)
{
    // Configuración del GPIO para el botón
    gpio_config_t io_conf_button = {
        .intr_type = GPIO_INTR_DISABLE,
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << CONFIG_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en = GPIO_PULLUP_ENABLE
    };
    gpio_config(&io_conf_button);

    // Configuración del GPIO para el LED
    gpio_config_t io_conf_led = {
        .intr_type = GPIO_INTR_DISABLE,
        .mode = GPIO_MODE_OUTPUT,
        .pin_bit_mask = (1ULL << LED_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en = GPIO_PULLUP_DISABLE
    };
    gpio_config(&io_conf_led);

    uint8_t gpio_value;
    bool was_pressed = false;

    while (1)
    {
        gpio_value = gpio_get_level(CONFIG_GPIO_PIN);

        if (gpio_value == 0 && !was_pressed)
        {
            ESP_LOGI(TAG, "Botón pulsado!");
            was_pressed = true; // Marca el botón como presionado
            gpio_set_level(LED_GPIO_PIN, 0); // Apaga el LED
        }
        else if (gpio_value == 1)
        {
            if (was_pressed) {
                was_pressed = false; // Reinicia el estado cuando el botón se libera
                gpio_set_level(LED_GPIO_PIN, 1); // Enciende el LED
            }
        }

        vTaskDelay(10 / portTICK_PERIOD_MS);
    }
}

```

Los cambios realizados son:

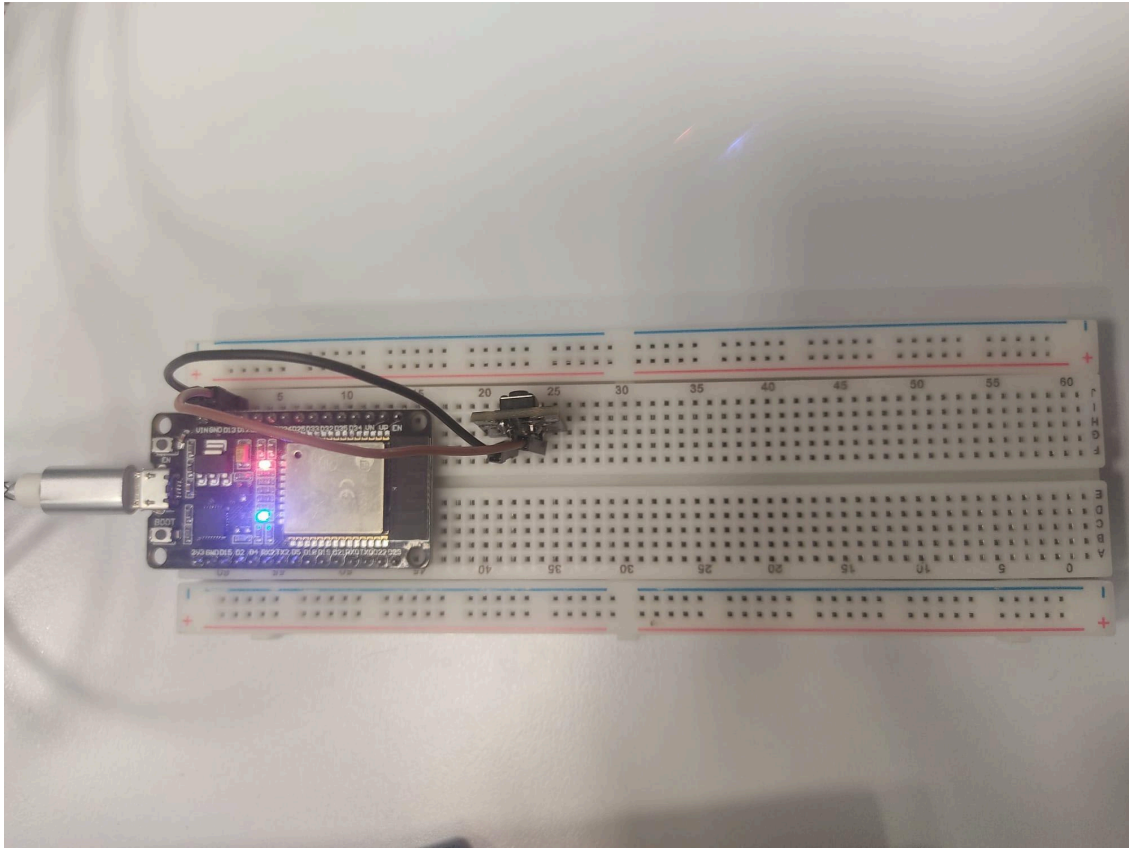
1. Configuración del GPIO para el LED:

- Se añade una configuración para el GPIO donde está conectado el LED (`LED_GPIO_PIN`), definiéndolo como salida (`GPIO_MODE_OUTPUT`). Esto permite controlar el LED al establecer el pin en alto o bajo.

2. Control del LED según el estado del botón:

- Cuando se detecta que el botón está presionado y `was_pressed` es `false`, se registra el evento, se establece `was_pressed` en `true` y se apaga el LED mediante `gpio_set_level(LED_GPIO_PIN, 0)`.
- Cuando el botón se libera (`gpio_value` es `1`), y después de haber estado presionado (`was_pressed` es `true`), se reinicia `was_pressed` a `false` y se enciende el LED con `gpio_set_level(LED_GPIO_PIN, 1)`.

De esta forma el circuito se queda con el LED encendido y se apaga al pulsar el botón.



Si ejecutamos el funcionamiento es el esperado:

```
es[0;32mI (331) gpio: GPIO[2]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 0| Intr:0 es[0m
es[0;32mI (4351) P1_3: Botón pulsado!es[0m
es[0;32mI (5281) P1_3: Botón pulsado!es[0m
es[0;32mI (6451) P1_3: Botón pulsado!es[0m
es[0;32mI (8281) P1_3: Botón pulsado!es[0m
es[0;32mI (9391) P1_3: Botón pulsado!es[0m
es[0;32mI (10461) P1_3: Botón pulsado!es[0m
es[0;32mI (11491) P1_3: Botón pulsado!es[0m
es[0;32mI (12571) P1_3: Botón pulsado!es[0m
```

2.- Haz lo mismo con interrupciones modificando el código src/interrupt/main.c. Aporta código, explicación y capturas en la memoria

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "esp_err.h"

const char* TAG = "P1_3";
#define LED_GPIO_PIN 2 // Definición del pin del LED

// Manejador de interrupción (ISR)
static void IRAM_ATTR gpio_isr_handler(void* arg)
{
    // Leer el estado actual del botón
    int gpio_state = gpio_get_level(CONFIG_GPIO_PIN);

    if (gpio_state == 0) {
        // Botón está presionado
        gpio_set_level(LED_GPIO_PIN, 0); // Apaga el LED
        ESP_DRAM_LOGI(TAG, "Botón pulsado, LED apagado");
    } else {
        // Botón está liberado
        gpio_set_level(LED_GPIO_PIN, 1); // Enciende el LED
        ESP_DRAM_LOGI(TAG, "Botón liberado, LED encendido");
    }
}

void app_main(void)
{
    // Configuración inicial del GPIO para el botón
    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_ANYEDGE, // Interrupción en ambos flancos
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << CONFIG_GPIO_PIN),
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .pull_up_en = GPIO_PULLUP_ENABLE
    };
    ESP_ERROR_CHECK(gpio_config(&io_conf));

    // Configuración del GPIO para el LED
    gpio_set_direction(LED_GPIO_PIN, GPIO_MODE_OUTPUT);
    gpio_set_level(LED_GPIO_PIN, 1); // Inicialmente el LED está encendido

    // Instala el servicio de ISR y añade el manejador
    ESP_ERROR_CHECK(gpio_install_isr_service(ESP_INTR_FLAG_SHARED));
    ESP_ERROR_CHECK(gpio_isr_handler_add(CONFIG_GPIO_PIN, gpio_isr_handler, NULL));

    while (1)
    {
        vTaskDelay(portTICK_PERIOD_MS * 1000); // Bucle de espera con delay más largo
    }
}

```

Explicación del Código Modificado:

1. ISR (Interrupt Service Routine):

- En el ISR, ahora se lee el estado del botón con ``gpio_get_level(CONFIG_GPIO_PIN)``.
- Si el botón está presionado (estado 0), el LED se apaga.
- Si el botón está liberado, el LED se enciende.
- Esto asegura que el LED refleje correctamente el estado del botón en tiempo real, apagándose cuando se presiona y encendiéndose cuando se libera.

2. Configuración de Interrupts:

- Se mantiene la configuración para interrumpir en ambos flancos (``GPIO_INTR_ANYEDGE``), ya que necesitamos detectar tanto el presionar como el soltar del botón.

El circuito queda igual y al ejecutar nos sale:

```
es[0;32mI (322) gpio: GPIO[13]| InputEn: 1| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:3 es[0m
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
I P1_3: Botón pulsado, LED apagado
I P1_3: Botón liberado, LED encendido
```